SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL1- Constraint-Based Problem Solving**
Weightage:	**40%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	S Susmitha	**Reg.No.:**	192571068
Department:	BTech CSE BioSciences	**Date:**	27.08.2026

ANNEXURE A

Constraint-Based Problem Solving

1.Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2.Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3.Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4.Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5.Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μs
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I S Susmitha (192571068) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Name	Register Number	Course Code	Course
S Susmitha	192571068	CSA1202	Computer Architecture

Question 1

Construct an I/O communication mechanism for a real-time embedded system with the given constraints (CPU utilization < 15%, sensors interrupting every 1 ms, vectored interrupt handling, and a choice between memory-mapped and I/O-mapped I/O for optimal latency).

Problem Understanding

The system has to service several sensors that each raise an interrupt roughly once every millisecond, while keeping the CPU free for at least 85% of the time. This means the I/O mechanism has to be interrupt-driven rather than polled, and the interrupt path itself has to be as short as possible so it does not eat into the CPU budget.

Constraint Analysis

- CPU utilization < 15% — rules out polling; interrupt-driven access is mandatory since polling would waste cycles checking sensors that have no new data.
- 1 ms interrupt rate per sensor, multiple sensors — the interrupt controller must resolve which device interrupted quickly, without the CPU having to poll a status register chain.
- Vectored interrupt handling requirement — each device is given a unique vector pointing straight to its ISR address, removing the need for a software search through a generic handler.
- Choice of memory-mapped vs I/O-mapped I/O — must be decided based on which gives lower per-access latency.

Proposed Solution

Memory-mapped I/O (MMIO) is selected. Device registers are mapped into the normal system address space, so the CPU accesses them with ordinary load/store instructions instead of dedicated IN/OUT instructions. This removes the extra instruction-decode step that I/O-mapped I/O needs, and lets the same addressing, pointer arithmetic and (where safe) compiler optimizations used for memory apply to device access as well — directly helping the latency requirement.

Application of Concepts

Two CO5 concepts are combined here: vectored interrupt handling (fast dispatch through an Interrupt Vector Table instead of a linear ISR search) and memory-mapped I/O (unified addressing for device

registers). Together they minimize both the time to reach the correct ISR and the time to actually read/write the device once inside it.

Justification

Vectored dispatch keeps ISR entry overhead low even when interrupts arrive as often as every 1 ms per sensor, and MMIO keeps each register access as cheap as a normal memory access. Together these keep the total time the CPU spends per interrupt small enough to satisfy the sub-15% utilization target, while still meeting the latency needs of a real-time system.

Architecture Diagram

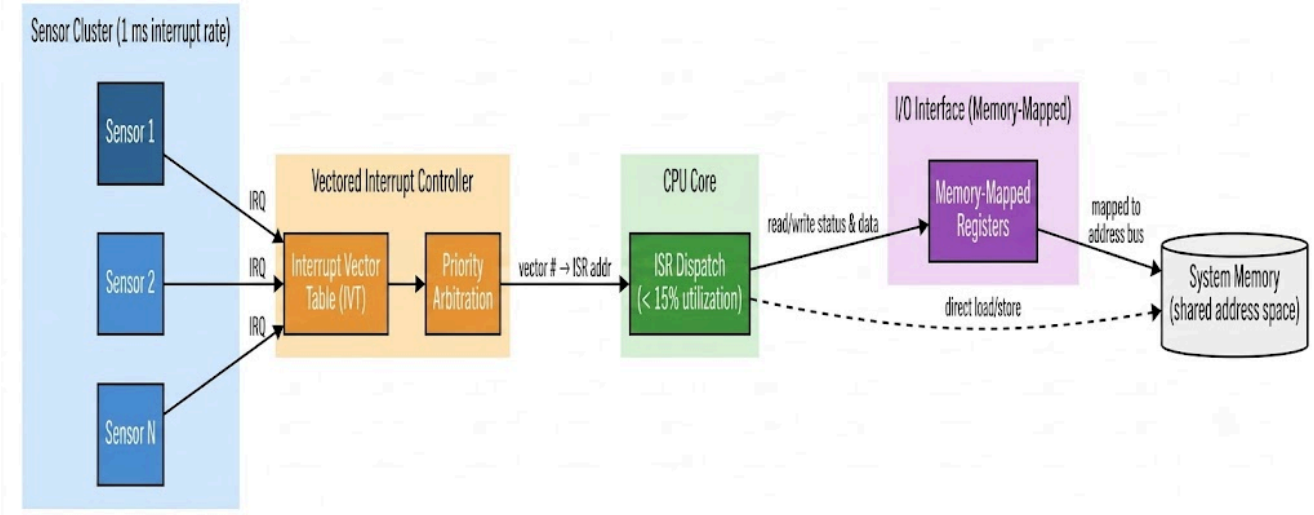

Fig. 1 — Sensors raise vectored IRQs; the vector table dispatches directly to the ISR, which accesses device registers through memory-mapped I/O.

Question 2

Design an I/O subsystem for high-speed storage (throughput > 3 GB/s) using NVMe over PCIe, where the CPU must not handle bulk data transfer, and a DMA controller is integrated with interrupt notification.

Problem Understanding

The subsystem must sustain more than 3 GB/s of storage throughput while keeping the CPU completely out of the actual data-movement path — the CPU should only issue commands and receive a notification once the transfer finishes.

Constraint Analysis

- > 3 GB/s throughput — a single PCIe Gen3 x4 link already provides close to 4 GB/s, so PCIe is the natural transport; SATA/AHCI-class links cannot reach this figure.
- Must use NVMe over PCIe — NVMe replaces the single-queue AHCI model with up to 64K parallel submission/completion queues, which is what allows the link's bandwidth to actually be used.
- CPU must not handle bulk transfer — the actual byte movement between the SSD and main memory has to be delegated to a DMA controller.
- DMA must be integrated with interrupt notification — the CPU needs to know when the transfer is done without polling.

Proposed Solution

The CPU places a command in an NVMe submission queue. The NVMe controller executes it over the PCIe link and hands the bulk data movement to a DMA engine, which transfers data directly between the SSD and main memory. Once the block is fully transferred, a completion queue entry is written and a single interrupt is raised to inform the CPU — at no point does the CPU itself copy data.

Application of Concepts

This answer applies PCIe (lane-based, scalable, point-to-point bus bandwidth), NVMe (a queue-based protocol purpose-built for flash storage on PCIe), and DMA with interrupt-based completion notification — all core CO5 topics — as a single integrated design rather than in isolation.

Justification

PCIe/NVMe is chosen over legacy AHCI/SATA because AHCI's single command queue becomes the bottleneck well before 3 GB/s. Offloading the transfer to DMA satisfies the 'CPU must not handle bulk data' constraint directly, and using one completion interrupt per block (instead of interrupting per unit of data) keeps context-switch overhead low, which matters even more as throughput rises.

Architecture Diagram

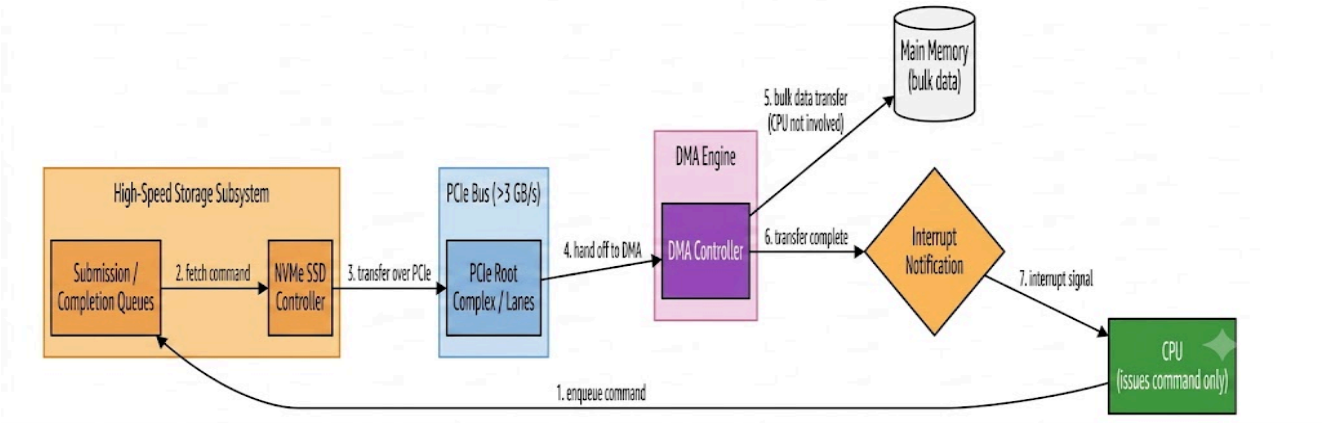

Fig. 2 — CPU issues a command; NVMe over PCIe and a DMA engine move the bulk data; the CPU is notified only on completion.

Question 3

Construct a multi-device communication architecture for 6 USB peripherals and 2 Thunderbolt devices, avoiding interrupt starvation, using both hardware and software interrupts, and ensuring fair bus arbitration.

Problem Understanding

Two device classes with different bandwidth and urgency profiles — six USB peripherals and two Thunderbolt (PCIe-tunneled) devices — must share the system's interrupt and bus resources such that no device is ever indefinitely denied service.

Constraint Analysis

- 6 USB peripherals — serviced through a USB host controller that itself schedules devices in frames/microframes.
- 2 Thunderbolt devices — higher bandwidth, PCIe-tunneled, and naturally tend to dominate bus time if given strict priority.
- Avoid interrupt starvation — a pure priority scheme (Thunderbolt always first) would starve the USB peripherals, so fairness has to be engineered in.
- Use both hardware and software interrupts — urgent, time-sensitive events should use hardware interrupts; routine/deferred work (status checks, housekeeping) should be pushed to software interrupts so the hardware interrupt path stays short.

Proposed Solution

Both the USB host controller and the Thunderbolt controller submit bus requests to a fair (weighted round-robin) arbiter rather than a fixed-priority one. The arbiter forwards urgent requests as hardware interrupts and routes deferred, non-time-critical work as software interrupts. Because the arbitration is round-robin rather than strict priority, every device is guaranteed a bounded maximum wait, which is what prevents starvation.

Application of Concepts

This design applies bus arbitration theory (round-robin vs. strict priority and its starvation implications), USB's frame-based device scheduling, Thunderbolt's PCIe tunneling, and the hardware-vs-software interrupt distinction together to meet a fairness constraint rather than a raw-throughput one.

Justification

Weighted round-robin arbitration is preferred over fixed priority specifically because fixed priority would let the two Thunderbolt devices monopolize the bus and starve the USB peripherals. Splitting servicing into hardware interrupts (for genuinely urgent events) and software interrupts (for everything else) also keeps each hardware ISR short, which further reduces the chance that a slow ISR delays the next device's turn.

Architecture Diagram

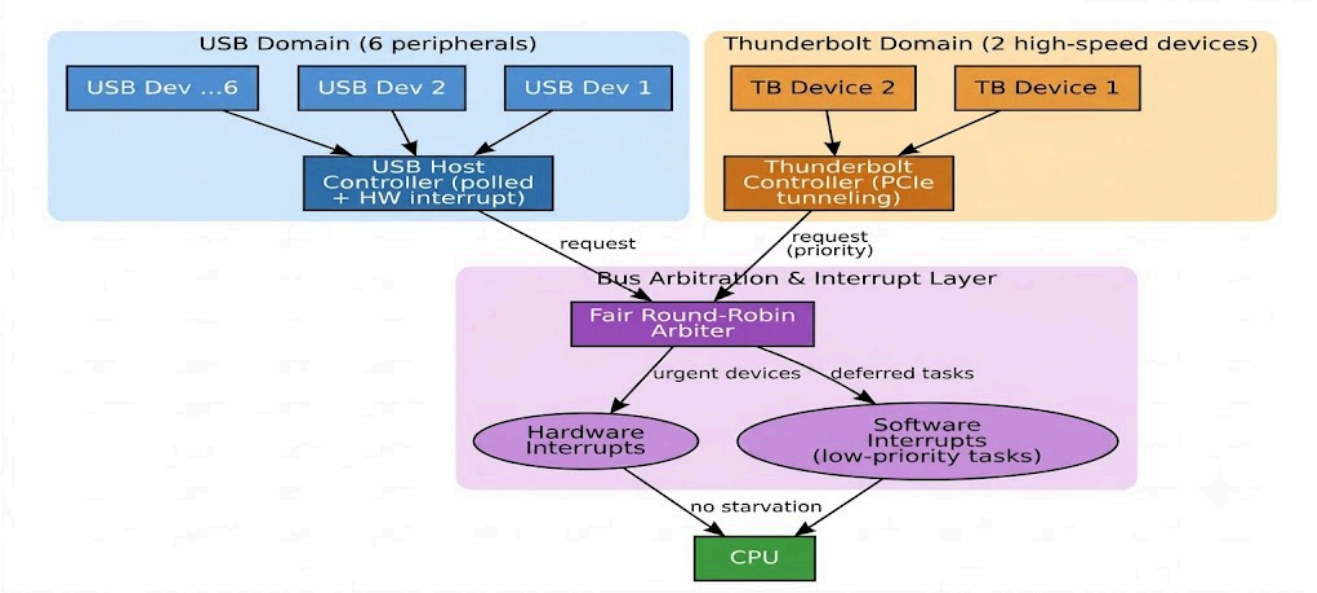

Fig. 3 — USB and Thunderbolt requests pass through a fair round-robin arbiter that splits servicing between hardware and software interrupts.

Question 4

Design a data acquisition system for a continuous 500 MB/s data stream with minimal CPU intervention, comparing DMA transfer against interrupt-driven I/O, and using memory-mapped I/O for device registers.

Problem Understanding

The acquisition front-end produces data continuously at 500 MB/s. The system must move this data into memory with as little CPU involvement as possible, which means the transfer mechanism itself has to be chosen carefully — not just the register interface.

Constraint Analysis

- 500 MB/s continuous stream — if the CPU had to be interrupted for every data unit, the interrupt rate alone would consume nearly all available CPU cycles.
- Minimal CPU intervention required — directly rules out programmed I/O and per-sample interrupt handling as the primary transfer mechanism.
- Must compare DMA vs interrupt-driven I/O — the design has to justify the choice quantitatively, not just assert it.
- Memory-mapped I/O for device registers — status/control registers should sit in the normal address space for simple, low-overhead access.

Proposed Solution

The device signals data-ready status through a memory-mapped register. This status triggers the DMA controller to move data in large blocks directly into a memory buffer, with the CPU only being interrupted once per completed block rather than once per sample. Pure interrupt-driven transfer, where the CPU is interrupted for every unit of data, is considered and rejected for the reasons below.

Application of Concepts

This applies DMA block/burst transfer, memory-mapped register access, and interrupt coalescing (batching many data events into one completion interrupt) — showing that the same 'interrupt vs DMA' trade-off from CO5 applies differently depending on data rate.

Justification

At 500 MB/s, even modest sample sizes correspond to a very high event rate — for example, 4-byte samples would imply well over 100 million interrupts per second, which is not achievable with typical ISR entry/exit overhead. DMA, by moving data in large blocks and interrupting only on block completion, reduces the interrupt rate to a handful per second, which is what actually makes 'minimal CPU intervention' achievable at this data rate.

Architecture Diagram

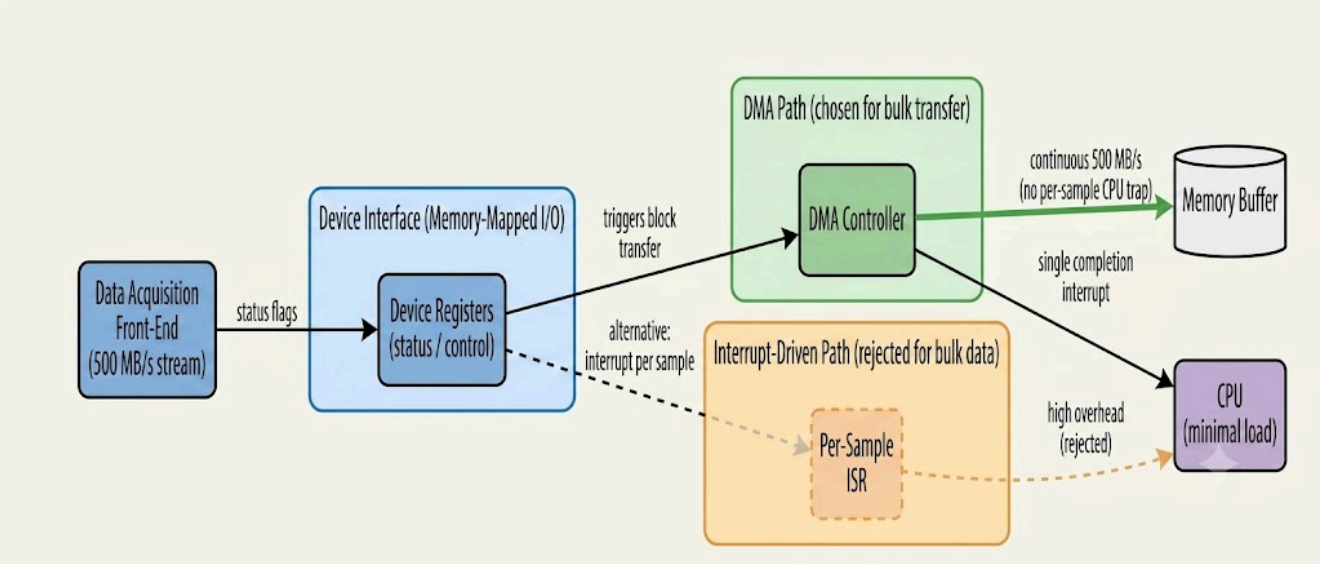

Fig. 4 — Device status is read via MMIO; the DMA path (solid) is selected over the rejected per-sample interrupt path (dashed).

Comparison: DMA vs Interrupt-Driven I/O

Parameter	**Interrupt-Driven I/O (per sample)**	**DMA-Based Transfer (block)**
CPU involvement	One ISR entry/exit for every sample	CPU issues one setup command; hardware moves the block
Interrupt rate	Extremely high at 500 MB/s (impractical)	One interrupt per completed block
Throughput ceiling	Limited by ISR overhead, not device speed	Limited by memory/bus bandwidth only
Suitability here	Rejected	Selected

Question 5

Construct an interrupt handling mechanism supporting nested interrupts with maximum latency under 10 μs, using vectored interrupts for high-priority devices and software interrupts for background tasks.

Problem Understanding

The mechanism must let a higher-priority interrupt preempt an ISR that is already running (nesting), while guaranteeing that the worst-case time from interrupt assertion to ISR start stays under 10 μs, and must separate truly urgent work from background work.

Constraint Analysis

- Nested interrupts — requires priority levels and a controller/CPU combination that can re-enable higher-priority interrupts while a lower-priority ISR is still executing.
- Latency < 10 μs — the vector lookup and context-save on ISR entry must be fast and bounded; a slow generic dispatch chain would blow this budget.
- Vectored interrupts for high-priority devices — hardware jumps straight to the correct ISR address using a priority-ordered vector table, skipping any software search.
- Software interrupts for background tasks — non-urgent work (logging, statistics, deferred processing) is triggered via a software interrupt/trap so it runs at low priority and never blocks the real-time path.

Proposed Solution

High-priority devices interrupt through a priority-ordered vector table that dispatches directly to their ISR. If a higher-priority interrupt arrives while a lower-priority ISR (ISR1) is executing, the context is saved, the higher-priority ISR (ISR2) runs nested inside it, and control returns to ISR1 afterward. Background work is kept out of this path entirely by routing it through software interrupts instead.

Application of Concepts

This combines interrupt priority levels and nesting, vectored dispatch, ISR context save/restore overhead, and software interrupts/traps for deferred work — the full nested-interrupt model from CO5, rather than a single flat interrupt queue.

Justification

Because vectored dispatch skips the software search a non-vectored system would need, and because only a minimal context is saved on entry, the entry latency for a high-priority device can be kept in the sub-microsecond to low-microsecond range — leaving margin under the 10 μs bound even when a nested preemption occurs. Moving background work to software interrupts ensures the hardware interrupt path is never held up by non-urgent processing, protecting the latency guarantee.

Architecture Diagram

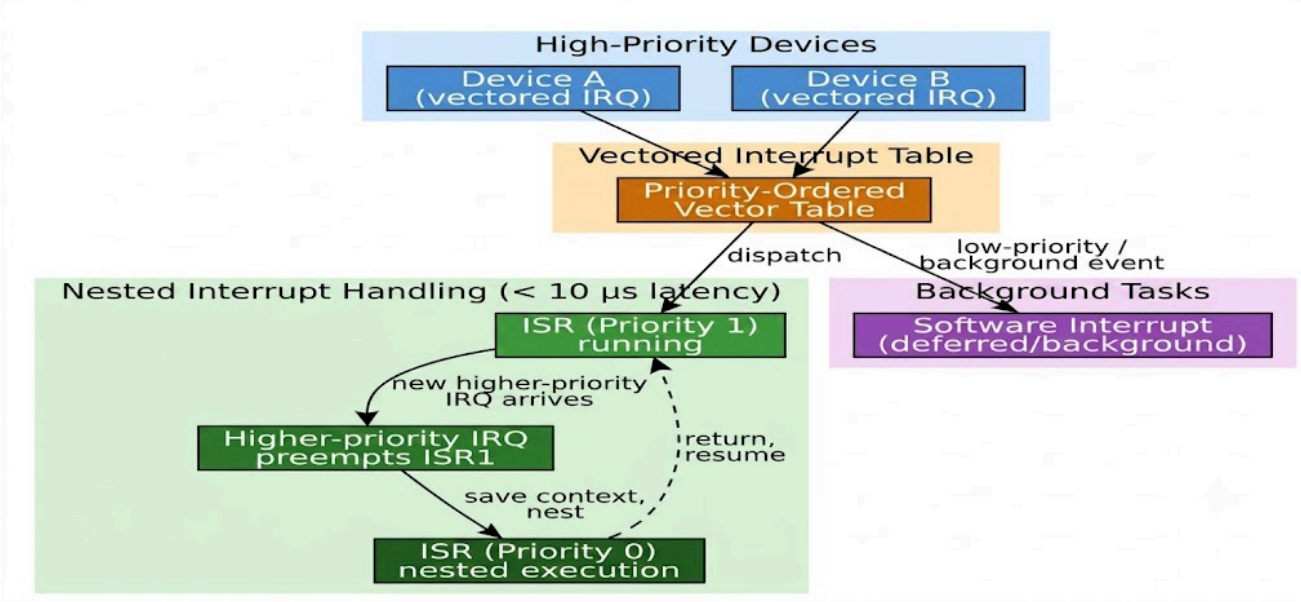

Fig. 5 — A priority-ordered vector table dispatches directly to ISRs; a higher-priority IRQ nests inside a running lower-priority ISR, while background work is deferred to software interrupts.